Code No: 136AQ **R16**

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
B. Tech III Year II Semester Examinations, August/September - 2024
COMPILER DESIGN
(Common to CSE, IT)

Time: 3 Hours **Max. Marks: 75**

Note: i) Question paper consists of Part A, Part B.
ii) Part A is compulsory, which carries 25 marks. In Part A, Answer all questions.
iii) In Part B, Answer any one question from each unit. Each question carries 10 marks and may have a, b as sub questions.

Used Paper Aug-2024

PART – A

(25 Marks)

1.a)	What is the role of the lexical analyzer in a compiler?	[2]
b)	State and briefly explain the need for a language translator.	[3]
c)	What are the benefits of LR parsers over LL parsers?	[2]
d)	Compare and contrast Top down parsing and bottom up parsing techniques.	[3]
e)	Define context-free grammars.	[2]
f)	Define synthesized attributes and inherited attributes.	[3]
g)	What are the key components of a run-time environment?	[2]
h)	How to identify a leader in a basic block provide example?	[3]
i)	Define a flow graph.	[2]
j)	Write about IN and OUT statements in data-flow analysis.	[3]

PART – B

(50 Marks)

2. Create a flowchart illustrating the process of lexical analysis, including input buffering, token recognition, and the role of a lexical analyzer. [10]

OR

3. Design a regular expression and corresponding finite automaton to recognize valid identifiers in a programming language, considering alphanumeric characters and underscores, ensuring that the automaton is deterministic. [10]

4. Evaluate the significance of LR parsing in compiler construction, especially in handling ambiguous grammars and improving parsing efficiency. Provide examples to support your argument. [10]

OR

5. Develop a context-free grammar for a simple arithmetic expression language, including rules for addition, subtraction, multiplication, and division. Illustrate the grammar with production rules and example expressions. [10]

6. Design a Syntax-Directed Translation Scheme (SDTS) to convert infix arithmetic expressions into postfix notation. Include semantic actions and attribute values to annotate the syntax tree. [10]

OR

7. Develop a set of semantic rules to perform type checking in a programming language compiler. Consider different types (e.g., integer, floating-point, boolean) and type compatibility rules. [10]

8. Analyze the advantages and disadvantages of stack allocation versus heap management in run-time environments, considering factors such as memory efficiency, access time, and fragmentation. [10]

OR

9. Evaluate the importance of garbage collection in modern programming languages. Discuss different garbage collection algorithms and their suitability for different application scenarios. [10]

10. Apply the concept of constant propagation to optimize the following code snippet: [10]

```
int x = 5;
int y = x + 3;
int z = y * 2;
```

OR

11. Analyze the importance of partial-redundancy elimination in compiler optimization. Discuss how it differs from other optimization techniques and its impact on code quality. [10]

---ooOoo---

Used paper Aug-2024